

Laborator 3.

Proiectarea sistemului de operare în timp real MyOS. Task-ul. Comutarea în funcție de timp

A. Caracteristicile sistemului de operare în timp real

1. Resursa pentru un sistem de operare în timp real

Prin termenul resursa se desemnează, în general, orice element de care are nevoie un task pentru a putea fi rulat.

Natura resurselor poate fi atât materială, cât și logică. Resursele de natură materială se numesc resurse materiale, iar resursele de natură logică se numesc resurse logice.

Exemple de resurse materiale sunt:

- procesorul;
- memoria internă;
- memoriile externe;
- dispozitivele de introducere/extragere;
- ceasul de timp real;
- dispozitivele speciale (procesoare specializate etc.).

Exemple de resurse logice sunt:

- subrutinele;
- tampoanele de date;
- fișierele;
- variabilele.

Resursele accesabile doar de către un singur task se numesc resurse locale. Acestea sunt mai ales de natura logică: subrutine, tampoane, fișiere, variabile, dar pot fi și de natură materială: dispozitive de introducere/extragere etc.

Resursele accesabile de către cel puțin două taskuri se numesc resurse comune. Și acestea pot fi atât de natură logică: subrutine, tampoane etc., cât și de natura materială: procesor, memorie, dispozitive de introducere/extragere etc.

Unele resurse comune pot fi utilizate deodată de către mai multe taskuri, altele nu. De exemplu: procesorul nu poate fi utilizat la un moment dat decât de un singur task.

2. Secțiune critică

O secțiune critică de cod, numita de asemenea regiune critică, este codul care trebuie tratat indivizibil. Din momentul în care secțiunea de cod începe să fie executată, nu trebuie să fie întreruptă.

Pentru a asigura acest lucru, de obicei întreruperile sunt dezactivate înainte de executarea codului critic și activate la sfârșitul codului critic.

3 Resursa partajata

O resursă partajată este o resursă care poate fi folosită de mai multe taskuri. Fiecare task trebuie să câștige acces exclusiv la resursa partajată pentru a evita coruperea datelor. Aceasta tehnica se numește excludere mutuală.

4 Multitasking

Multitasking este procesul de administrare și comutare al CPU (Central Processing Unit) între mai multe taskuri ; un singur CPU este comutat între mai multe task-uri secvențiale. Multitasking maximizează utilizarea procesorului și de asemenea asigură construirea modulară a aplicațiilor. Unul din aspectele importante ale multitasking-ului este faptul că permite programatorului să manipuleze complexitatea aplicațiilor în timp real. Aplicațiile sunt astfel mai ușor de proiectat și întreținut dacă se folosește multitasking.

5 Task

Un task, numit de asemenea și thread, este un program simplu care consideră că deține procesorul doar pentru el.

Procesul de proiectare pentru o aplicație în timp real implică divizarea sarcinii de lucru în mai multe taskuri care sunt responsabile de o porțiune a problemei.

Fiecărui task îi este acordată o prioritate, setul propriu de regiștrii CPU și propria zonă de stivă. Fiecare task este de regulă o buclă infinită.

Stările taskurilor:

Starea necreat (dormant)

Un task în starea necreat este un task necunoscut de executiv. Într-un sistem ce dispune de memorii externe, taskurile în starea necreat nu sunt încărcate în memorie și nu au alocate zone de cod, date și stivă. Într-un sistem fără memorii externe, taskurile în starea necreat au codul încărcat în memorie, dar nu dispun de zone de date și stivă. Nimic nu împiedică preferarea acestei situații și pentru cazul sistemelor cu memorii externe. Singura tranziție posibilă din starea necreat este către starea creat.

Starea creat

Un task în starea creat este un task cunoscut de executiv, având atribuit un identificator și alocate zonă de cod și zonă de stivă. Tranzițiile posibile din starea creat sunt către stările gata, respectiv necreat.

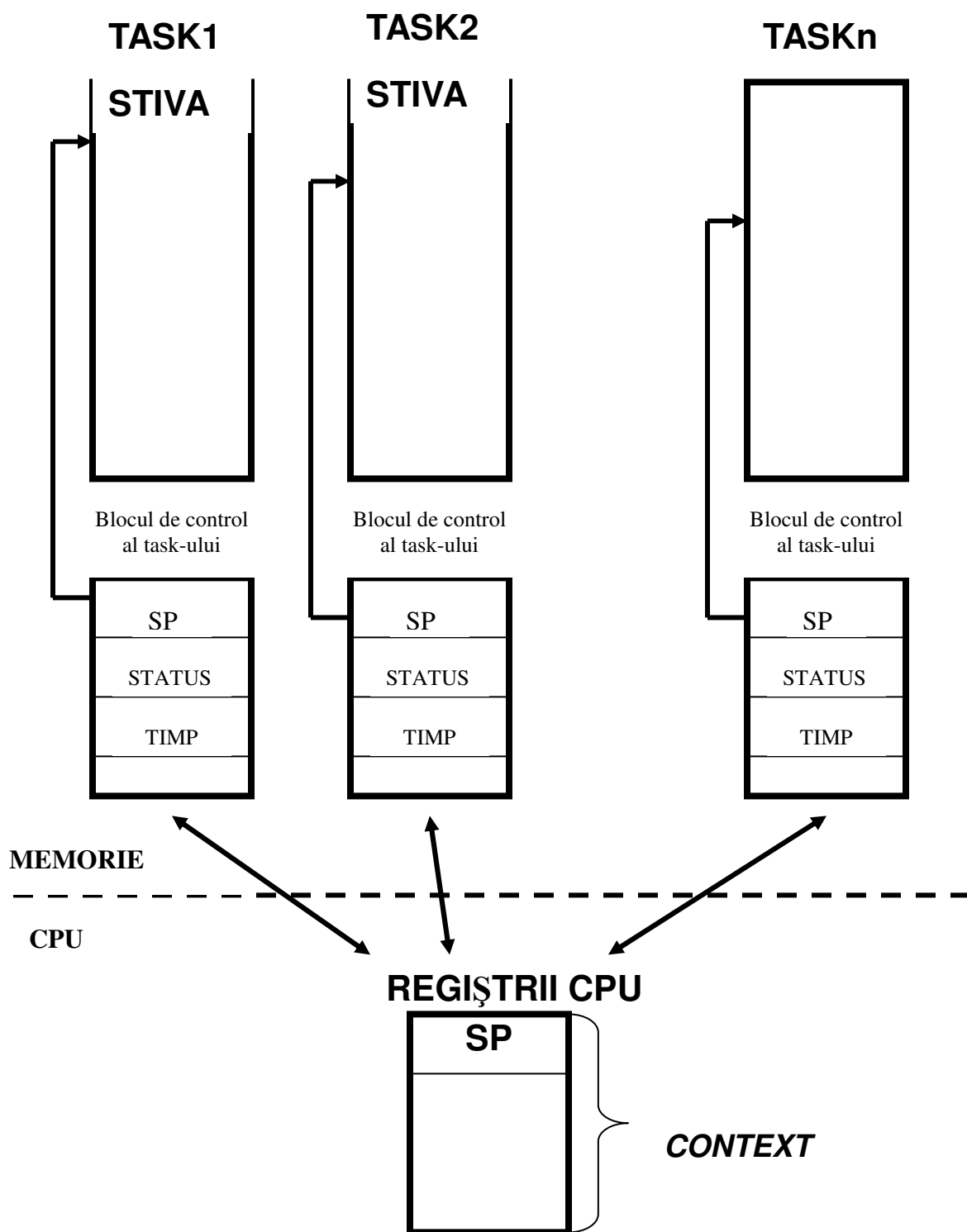


Figura 1 – Task-uri multiple

Starea gata (ready)

Un task în starea gata este un task apt de rulare. Lansarea sa nu depinde decât de prioritatea pe care o are în raport cu celelalte taskuri în starea gata și cu taskurile aflate în execuție.

Când un task în starea gata satisface condiția de prioritate, dispecerul îi atribuie procesorul, lansându-l în execuție de la începutul sau. Din starea gata, task-urile nu pot evolua decât către stările necreat, creat și în execuție.

Starea în execuție (running)

Un task în starea în execuție este un task a cărui rulare a început și care este încă apt de rulare, fie ca deține, fie ca nu, pentru moment, procesorul. Se spune despre un task aflat în aceasta stare ca este în execuție fizică dacă deține procesorul, și în execuție logică, altfel.

Când un task iese din execuție fizică prin pierderea controlului asupra procesorului, rămânând în execuție logică sau blocat, întregul sau context este salvat, pentru a putea fi refăcut, în vederea posibilității continuării rulării sale. Tranzițiile posibile din starea în execuție sunt către oricare dintre stările: necreat, creat sau blocat.

Nota: Taskurile aflate în stările în execuție și gata se numesc taskuri rulabile.

Starea blocat

Un task în starea blocat este un task care a fost în execuție, dar a devenit inapt de rulare, înainte de a se încheia, prin survenirea în cadrul lui a unei condiții de blocare. Cauzele blocării unui task sunt multiple: așteptarea unui mesaj, așteptarea unui eveniment, așteptarea disponibilității unei resurse critice, așteptarea ridicării unei suspendări, așteptarea expirării timpului de adormire, etc.

În momentul blocării unui task, întreg contextul său este salvat, pentru a putea fi refăcut, cu ocazia relansării sale fizice în execuție, ca urmare a dispariției condiției de blocare.

6 Comutarea taskurilor

Prin noțiunea de comutare a taskurilor, este desemnat ansamblul de acțiuni prin care se asigură ieșirea din rulare a unui task și intrarea în rulare a altui task; în anumite situații, este posibil ca un task să iasă din rulare și tot el să fie cel care intră în rulare, în cadrul unui aceluiași proces de comutare.

Comutarea taskurilor este realizată în cadrul unei funcții a executivului numită dispecer de taskuri (în engleză : scheduler).

Ea comportă următoarele acțiuni:

- salvarea contextului taskului care iese din rulare;
- determinarea taskului care urmează să intre în rulare;
- restaurarea contextului taskului care urmează să intre în rulare și, implicit, introducerea în rulare a acestui task.

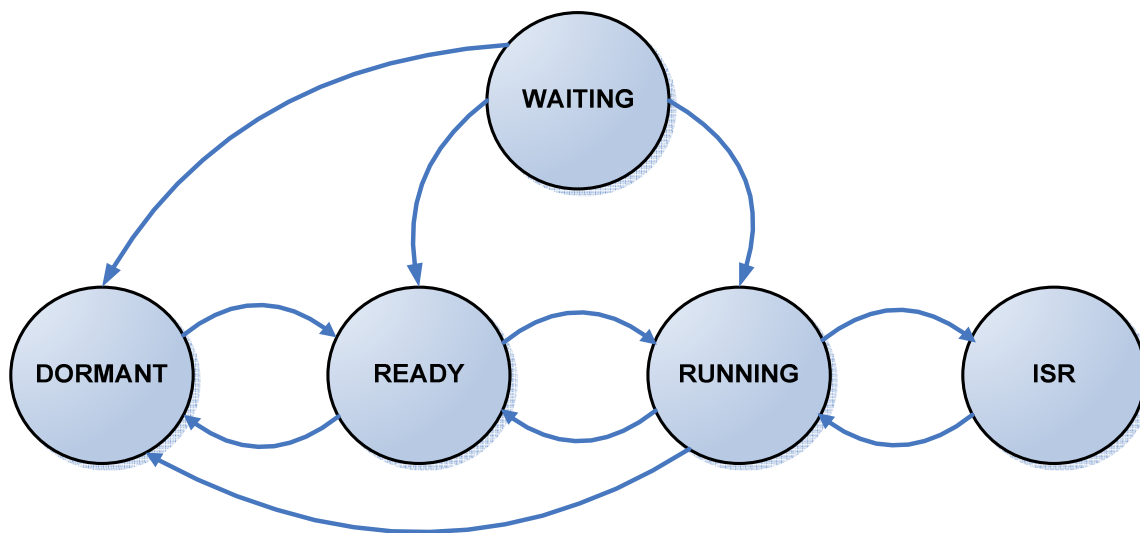


Figura 2 - Evoluția taskurilor în spațiul starilor

Contextul unui task este reprezentat printr-un ansamblu de informații absolut necesare pentru ca rularea lui să poată decurge corect în cadrul sistemului multitasking, indiferent în câte reprize ea se produce și indiferent ce altceva execută procesorul în intervalele de timp dintre respectivele reprize. Cantitatea și semnificația informațiilor care constituie contextul unui task sunt dependente de complexitatea executivului. Independent de aceasta, însă, câteva informații constituie, prin forța lucrurilor, nucleul ireductibil al oricărui context.

Aceste informații sunt:

- adresa instrucțiunii cu care taskul urmează să-și înceapă sau să-și continue rularea;
- cuvântul de stare a programului;
- conținuturile registrelor procesorului-alte decât cele implicate în pointarea instrucțiunilor, găzduirea cuvântului de stare a programului și pointarea stivei;
- adresa curentă în stiva taskului;
- starea curentă a taskului.

7 Nucleu (kernel)

Kernelul este partea sistemului multitasking responsabilă cu administrarea taskurilor (adică administrarea timpului procesorului) și comunicația între taskuri. Serviciul principal oferit de kernel este comutarea contextului.

Utilizarea unui kernel în timp real va simplifica în general proiectarea sistemului permițând aplicației să fie împărțită în mai multe taskuri administrate de kernel. Un kernel va încărca sistemul deoarece are nevoie de memorie ROM (spațiu pentru cod), memorie RAM pentru structurile de date și cel mai important fiecare task necesită propria stivă ceea ce duce la consumarea de memorie RAM foarte rapid. De asemenea, un kernel consumă și timp de procesor (de regulă între 2 și 5%).

Un kernel permite folosirea mai bună a procesorului punând la dispoziție servicii indispensabile cum ar fi managementul semafoarelor, căsuțe de mesaje, cozi, time delays, etc.

B Contorizarea timpului și comutarea task-urilor în funcție de timp

Unul din primele obiective care ni le propunem, în cadrul proiectării unui sistem de operare în timp real, îl reprezintă construirea modulului care se ocupă cu contorizarea timpului.

Contorizarea timpului este utilizată, în primul rând, pentru determinarea cuantei de timp în ritmul căruia se va face partajarea resurselor în funcție de timp. Cuanta de timp se numește în engleză tick-time. Tehnica prin care resursele sunt partajate între task-uri un interval de timp se numește time-sharing. Practic în funcționarea unui sistem de operare în timp real, aproximativ 90% din comutări între task-uri sunt determinate de expirarea timpului alocat pentru fiecare task și numai 10% de evenimente. Desigur, motivul pentru care este proiectat un sistem în timp real este ca acesta să răspundă cât mai prompt la evenimente dar partajarea în funcție de timp reprezintă o tehnică foarte folosită prin care se asigură faptul că sistemul este pregătit să răspundă la evenimente.

Din acest motiv, o primă etapă în proiectarea unui sistem de operare în timp real este configurarea canalelor de tip timer-counter de care dispune sistemul. Folosirea circuitelor periferice specializate timer-counter prezintă avantajul economiei utilizării unității centrale de calcul. Între timp aceasta poate fi utilizată la alte sarcini, care să ruleze în paralel. Prezența a cel puțin unui canal specializat timer-counter este obligatorie într-un sistem în timp real.

În plus, contorizarea timpului este folosită în funcții de sincronizare cu unele resurse hardware, cum ar fi de exemplu funcțiile de temporizare pentru tratarea unor tastaturi sau pentru afișarea pe ecrane. Același avantaj al eficientizării utilizării CPU apare și prin utilizarea temporizării hardware.

Pentru evitarea unei interogări permanente în buclă s-a mers pe direcția generării unor cereri de întreruperi la epuizarea intervalului de timp. Lucrul cu întreruperi, folosind canalul timer-counter este poate tot atât de necesar precum este lucrul cu timer-ul însuși. Evident că în lipsa lucrului cu întreruperi s-ar pierde toată eficiența în alocarea resurselor CPU care o dă utilizarea canalul timer-counter deoarece timpul pierdut prin interogarea (polling) indicatorului sfârșit numărare de la timer-counter este aproape același cu cel care ar fi fost pierdut în cazul realizării unei contorizări software.

În cele ce urmează se vor prezenta mai multe probleme în care se utilizează canalele timer-counter ale microcontrolerului.

Problema 1. Un MCU AVRATMega128 este conectat la un ceas extern cu frecvența de 16MHz. Să se realizeze, folosind canalul timer-counter 0 o temporizare de 1ms și apoi să se folosească aceasta într-o funcție de tipul delay. Parametrul funcției va fi numărul de milisecunde.

Soluție

În această problemă este ilustrat modul în care se configurează canalul timer la AVRATMega128.

După cum s-a arătat în laboratorul numărul 1, configurarea canalului timer se face în următorii pași:

1. se stabilește modul de lucru;
2. se stabilește sursa ceasului (temporizator sau numărător);
3. se stabilește valoarea registrului OCR dacă se lucrează în modul comparator, modulator PWM sau modulator PWM defazat și a registrului TCNT dacă se lucrează în modul normal;
4. se activează lucrul cu întreruperi dacă se dorește.

Pentru această problemă avem următoarele:

1. Se va lucra în modul normal de lucru. Aceasta înseamnă numărare în sens crescător cu generarea întreruperii la atingerea valorii maxime.
2. ceasul provine din cel al sistemului, întrucât în problemă nu se specifică alt ceas extern. Ceasul intern este deci de 16MHz. Pentru a realiza o temporizare de 1ms adică un semnal cu frecvența de 1000Hz este necesară divizarea semnalului de ceas cu:

$$d = 16000000/1000=16000.$$

Canalul 0 nu poate diviza decât cu maxim 256, fiind un canal pe 8 biți. Totuși fiecare canal timer-counter al lui ATMega128 posedă câte un prescaler programabil care realizează o predivizare a semnalului de ceas.

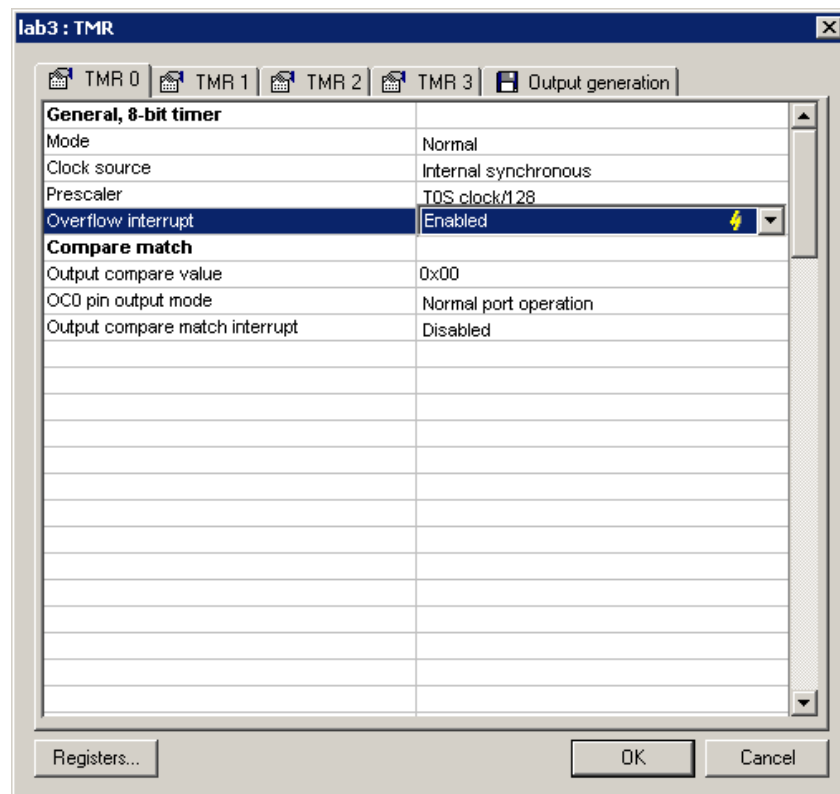
Pentru acest exemplu am ales un prescaler cu valoarea 128. Deci se va obține:

$$16.000.000 / 128 = 125.000 / 125 = 1.000$$

3. Valoarea registrului TCNT0 la fiecare întrerupere se va reîncărca cu:
 $256 - 125 = 131$, adică valoarea constantei de divizare pentru canalul 0.
4. Se va activa lucrul cu întreruperi la depășirea valorii maxime pe 8 biți de către valoarea din registrul TCNT0.

Etapele în rezolvarea problemei sunt următoarele (pentru detalieri revedeți laboratoarele 1 și 2!!)

1.În IAR MakeAPP se crează un proiect și se va introduce următoarea configurație legată de canalul 0:



Se generează fișierele librării (pentru detalii revezi laboratoarele 1 și 2).

2. Se creează în IAR un proiect nou la care se vor adăuga fișierul librărie `ma_tmr.c`.
Notă: fișierele librărie trebuie să fie situate în același director cu fișierele proiectului pentru a putea efectua incluziunile fără specificarea întregii căi.

3. Se creează un fișier numit `main.c`, care se va adăuga la proiect.

4. Codul sursă al fișierului `main.c` va fi următorul:

```
#include<inavr.h>
#include"ma_tmr.h"
```

```
int timp_ms;
void delay(int ms);
```

```

void main(void)
{
    MA_Init_TMR();

    MA_Start_TMR(0,0);

    __enable_interrupt();
    for(;;)
    {
        delay(12);
    }
}

void delay(int ms)
{
    timp_ms = ms;
    while(timp_ms);
}

```

Codul sursă adăugat la fișierul ma_tmr.c este:

La început, în partea declarativă:

```
extern int timp_ms;
```

iar în subrutina de tratare a întreruperii:

```

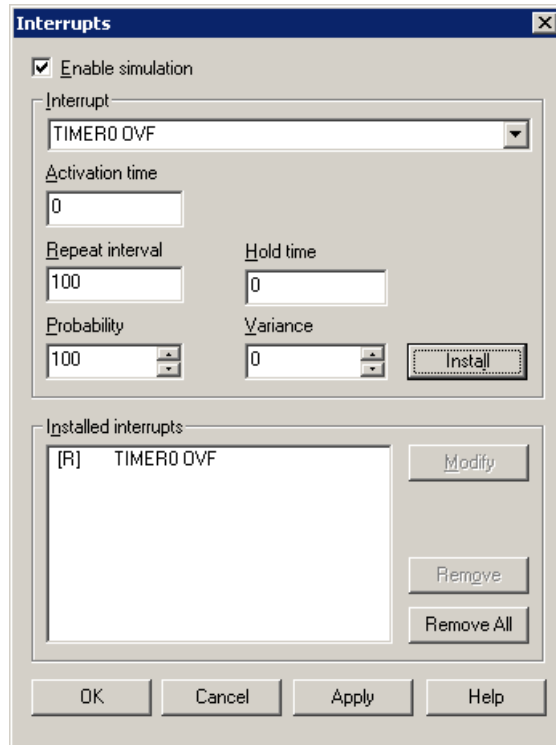
    TCNT0 = 131;
    if(timp_ms)
        timp_ms--;

```

Înainte de a se da compilarea se modifică fișierul ma_tmr.c astfel: se comentează incluziunea #include "usercode.h" și toate macrodefinițiile care încep cu ENTER_MA, INSIDE_MA sau EXIT_MA.

De asemenea se setează din meniul Project – Options – General options – Processor configuration valoarea cpu = m128.

5. Se va porni simularea și se va seta simulatorul pe generarea unei întreruperi externe cu frecvența de 100 unități probabilitatea 100% conform imaginii de mai jos:



6. Se vor introduce break point in interiorul subrutinei și se va monitoriza incrementarea variabilei timp_ms.

Întrebări:

1. Cine este TCNT0 și de ce este necesară încărcarea lui cu valoarea 131 la apelarea de fiecare dată a subrutinei de tratare a întreruperii la canalul timer-counter 0 ?
2. Să se realizeze un program care realizează întreruperi cu frecvența de 10ms.

Problema 2. Se dau trei task-uri finite t1, t2 și t3 (task finit- task care își execută într-un timp finit activitățile) și o zonă de memorie pe 100 de octeți, adresată prin intermediul unei variabile index. Fiecare task când este accesat scrie un octet cu codul acestuia (de exemplu t1 scrie codul 1, t2 scrie codul 2, t3 scrie codul 3) în zona de memorie pe poziția corespunzătoare index-ului și incrementează indexul. Dacă s-a ajuns la epuizarea spațiului în memorie execuția tuturor task-urilor va fi suspendată. Să se realizeze un program care administrează comutarea task-urilor la un interval de timp de 5ms. Frecvența procesorului este de 16MHz.

Soluție

Se va folosi canalul timer-counter 0 în modul normal de lucru (numărător cu incrementare).

Pentru realizarea unei temporizări de 5ms (=200Hz) ar trebui efectuată următoarea împărțire:

$$16.000.000 / 200 = 80.000.$$

Deci semnalul de 16M ar trebui divizat cu 80.000 pentru a obține frecvența dorită de 5ms. Cum canalul 0 timer-counter nu este suficient se va folosi prescaler-ul setat pe divizarea maximă. Constanta de divizare a canalului 0 timer-counter este dedusă din calculul de mai jos:

$$16.000.000 / 1024 = 15.625 / 200 \sim 78$$

$$\text{Deci constata de divizare va fi } 256 - 78 = 178.$$

Deci se va folosi următoarea configurație: prescaler 1024, constata divizare 178. Se vor activa întreruperile la depășire.

Pașii de realizare a problemei sunt următorii:

1. Se fac setările de mai sus în IARMakeAPP și se generează fișierele librării.
2. Se creează în IAR un proiect nou la care se vor adăuga fișierul librărie `ma_tmr.c`.
Notă: fișierele librărie trebuie să fie situate în același director cu fișierele proiectului pentru a putea efectua incluziunile fără specificarea întregii căi.
3. Se creează un fișier numit `main2.c`, care se va adăuga la proiect. Se va crea și un fișier `main2.h` în care se vor afla declarațiile funcțiilor care vor fi folosite în `ma_tmr.c`
4. Codul sursă al fișierului `main2.c` va fi următorul:

```
#include<inavr.h>
#include"ma_tmr.h"

//variabila index folosita pentru adresarea memoriei
char index=0;
//zona de memorie accesata de taskuri
char memorie[100];

//declararea functiilor task
void t1(void)
{
    if(index<100)
    {
        memorie[index]=1;//scrie codul task-ului
        index++;
    }
}
```

```

void t2(void)
{
    if(index<100)
    {
        memorie[index]=2;//scrie codul task-ului
        index++;
    }
}

```

```

void t3(void)
{
    if(index<100)
    {
        memorie[index]=3;//scrie codul task-ului
        index++;
    }
}

```

```

void main(void)
{
    //initializare timer
    MA_Init_TMR();
    MA_Start_TMR(0,0);
    //initializare intreruperi
    __enable_interrupt();

    //startare primul task
    t1();
    for(;;)
        ;

}

```

Codul sursă al fișierului main2.h va fi următorul:

//declaratii functii pentru a putea fi apelate in alte proiecte

```

void t1();
void t2();
void t3();

```

Codul sursă adăugat la fișierul ma_tmr.c este:

La început, în partea declarativă:

```
#include "main2.h"
extern char index;
extern char memorie[100];
//variabila care contorizeaza task-ul curent
char task_crt = 1;
```

iar în subrutina de tratare a întreruperii:

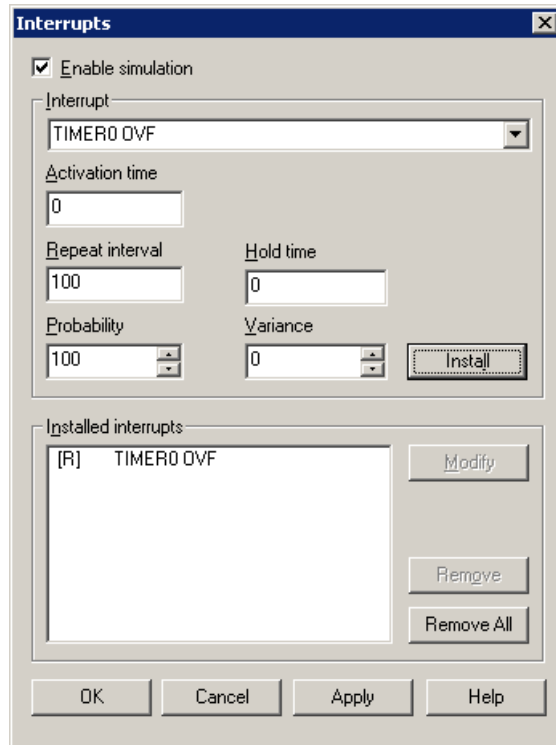
```
TCNT0= 178;

switch(task_crt)
{
case 1 : t2();
break;
case 2 : t3();
break;
case 3: t1();
break;
}
task_crt++;
if(task_crt==4)
task_crt=1;
```

Înainte de a se da compilarea se modifică fișierul ma_tmr.c astfel: se comentează incluziunea #include "usercode.h" și toate macrodefinițiile care încep cu ENTER_MA, INSIDE_MA sau EXIT_MA.

De asemenea se setează din meniul Project – Options – General options – Processor configuration valoarea cpu = m128.

7. Se va porni simularea și se va seta simulatorul pe generarea unei întreruperi externe cu frecvența de 100 unități probabilitatea 100% conform imaginii de mai jos:



8. Se vor introduce break point in interiorul subrutinei și se va monitoriza incrementarea variabilei timp_ms. În primul rând se va monitoriza unde este depus vectorul memorie folosind opțiunea Watch (click dreapta pe numele memorie). Apoi va fi afișat conținutul memoriei de date (View – memory – data). Se va observa plasarea „semnăturilor” fiecărui task și ordinea de plasare.

Întrebări și probleme:

1. Care este rolul variabilei task_crt ?
2. Care este rolul variabilei index ?
3. Să se efectueze modificările necesare astfel încât programul să execute task-urile în următoarea ordine: task2, task1, task3
4. Să se efectueze modificări în program astfel încât accesul la resursa memorie să fie unui circular. Astfel, când resursa memorie se umple să fie reluată scrierea de la început.
5. Să se realizeze un program care poate să administreze 5 task-uri diferite care accesează o memorie de 50 octeți. Fiecare task poate să scrie unul sau mai mulți octeți în memorie (de exemplu task 1 - 1 octet, task 2 – 2 ș.a.m.d). Să se realizeze un administrator de task-uri care să realizeze comutarea acestora la 1ms cu următoarea ordine t1-t3-t2-t5-t4 și cu blocare la umplerea memoriei

